

Introduction

DATA 236, Distributed Systems

Refresher Homework

- Practice JavaScript, docker, AWS
- Focus on new features
 - Arrow functions
 - Async-await
 - Promise

Projects

Projects are an opportunity to further discussions and exploration through practical experience.

2 Programming projects (Lab) and 1 group project

Programming projects—individual or pair

Class project: Team project design and implementation

Manage your time carefully

- Projects take 30-70 hours to complete
 - **Do not wait until the last week**
 - **Rather program every day**
- What you have to plan for
 - Project investigation
 - Understanding concepts and technologies
 - Implementation
 - Testing
 - Documentation

- | | | |
|----------------|---|------------|
| • Grade is not | ∞ | knowledge |
| • knowledge | ∞ | struggle |
| • knowledge | ∞ | Time spent |
| • Success | ∞ | knowledge |

Why the Projects Matter

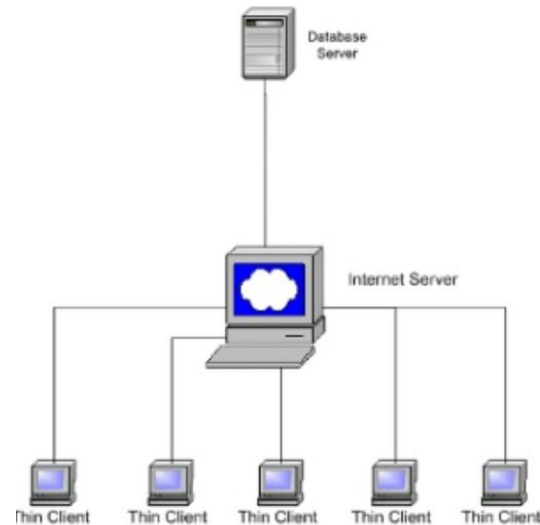
- From a student, after taking DATA 236:
 - "I recently accepted an offer for a Software Development Engineer internship at Amazon."
 - "Working on the Uber Eats prototype and the Uber Transport simulation projects was incredibly helpful. These projects not only solidified my understanding of distributed systems concepts but also provided me with concrete examples I was able to draw upon during Amazon's Leadership Principles behavioral round."
 - - Kush Bindal, MS Applied Data Intelligence
- The projects are not busywork. They are what you will talk about in interviews.

Web and Distributed Architecture:

Client Server Architecture

- A three-way interaction in a client/server environment
 - User interface is stored in
 - Business application logic is stored in
 - The data is stored in

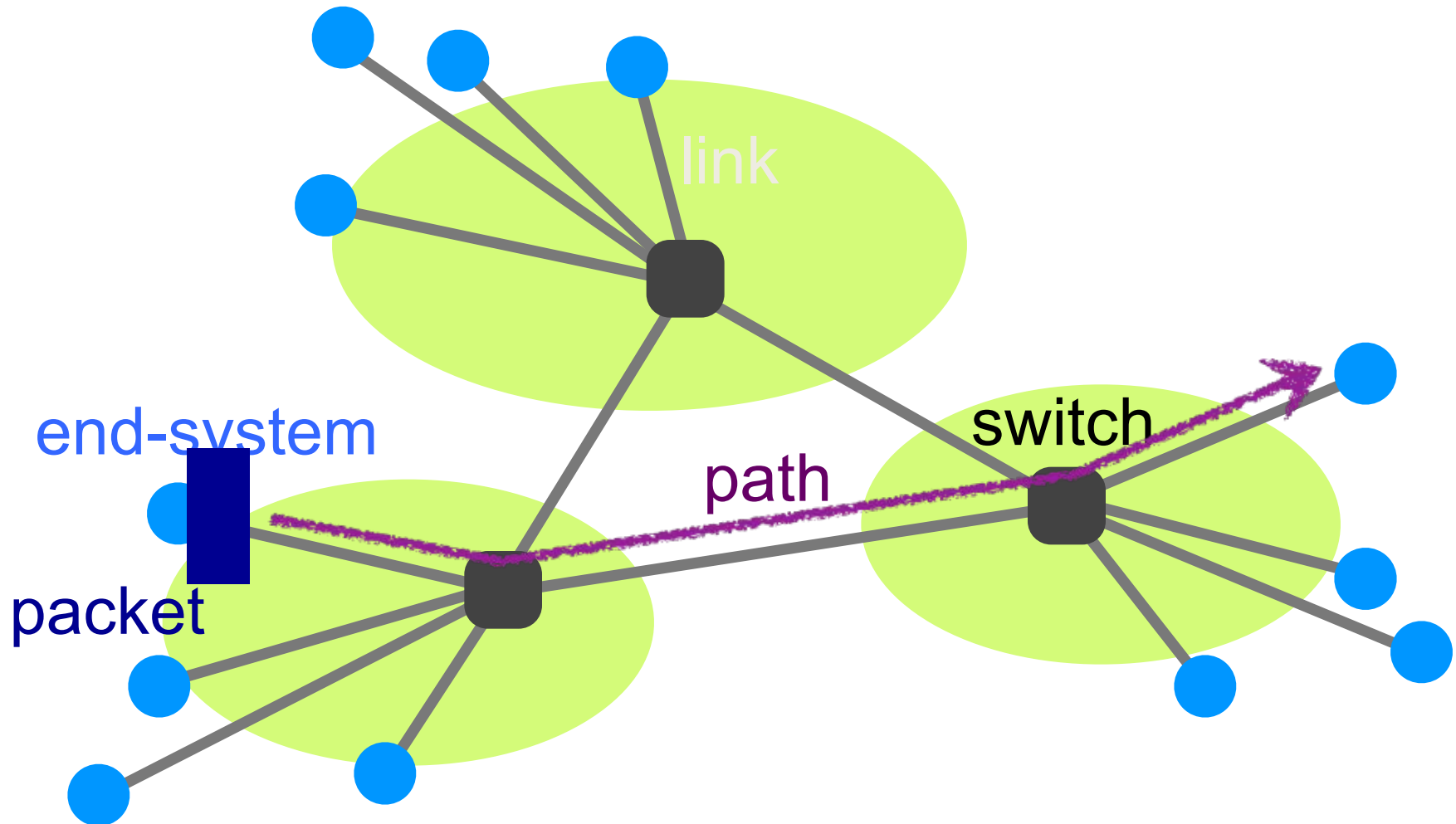
- Single Tier: “Mainframe” era
- Dual Tier: “personal Computer” era
- 3-Tier: “thin client”



Goals for the Internet

- Ability to connect many different networks
 - Ethernet and optical and Wifi and
- Ability to scale to entire world
- Ability to tolerate and recover from failures
- Able to support wide variety of applications

The central mechanism of Internet?



Tremendous scale

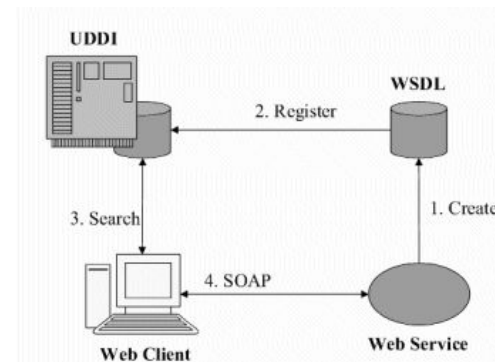
- 3.8 Billion users (51% of world population)
- 1.24 Trillion unique URLs (web pages)
- Every second, approximately
 - > 6,000 tweets are tweeted
 - > 40,000 Google queries are searched
 - > 2 million emails are sent

We use the phrase “Internet Scale” to refer to such systems

Web services

- HTTP based
- SOAP/UDDI/WSDL is a general framework (based on XML) for describing network services. Still runs in banking, government healthcare and telecom. UDDI is effectively obsolete and SOAP is rare in new projects.

-



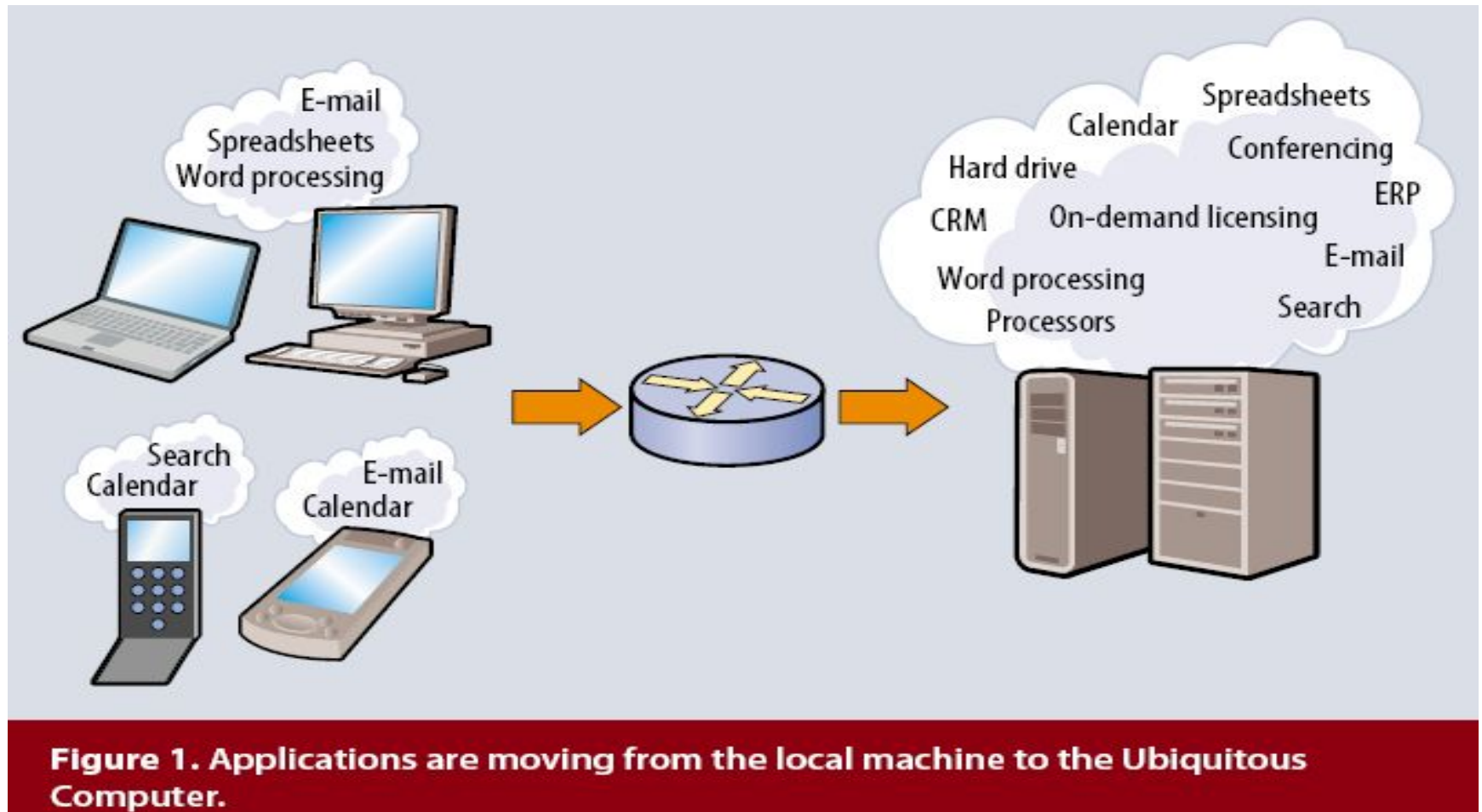
RESTful Services

- **Representational State Transfer (REST)**
 - Roy Fielding, 2000 (doctoral dissertation)
 - Examination of the Internet as a stateless service of near-limitless expansion model with a simple but effective information delivery system
- **Key concepts**
 - Resources - source of information
 - Consistent access to all resources
 - As in interface and communication – Not content or function
 - Stateless protocol
 - Hypermedia – links in the information to other data (connectedness)

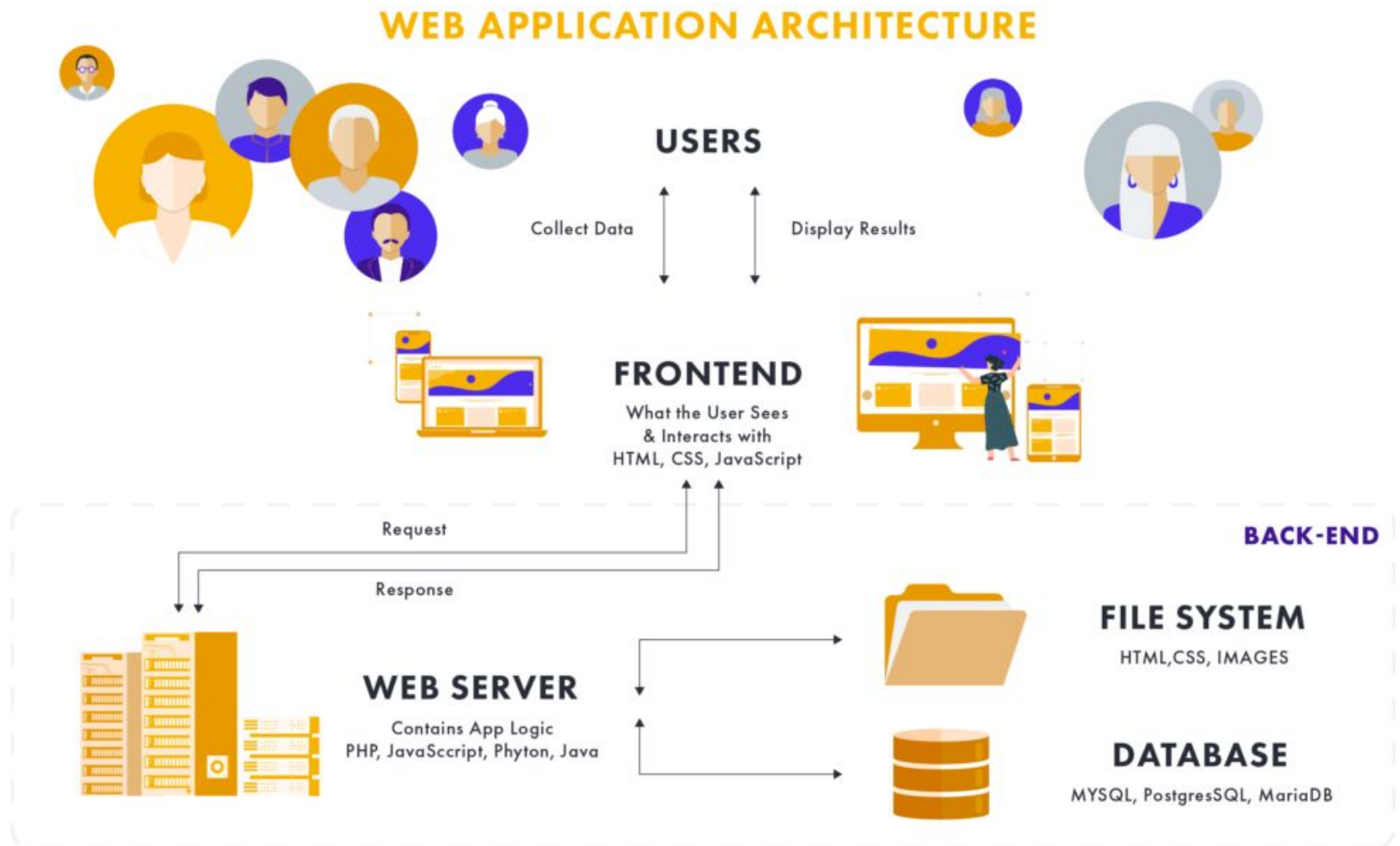
Designing a REST API

- Model resources as nouns, not actions
 - Good: GET /students/42/courses
 - Avoid: GET /getStudentCourses?id=42
- The method is the verb. The path is the thing.
- Practical rules
 - Plural collection names: /students, /courses
 - Version the API: /v1/students
 - Paginate every list endpoint: ?limit=50&offset=100
 - Return errors in a consistent JSON shape, never a raw stack trace
 - Filter and sort with query parameters, not new endpoints
- Document the contract with OpenAPI. FastAPI generates it for you at /docs - one of the reasons we use it in this course.

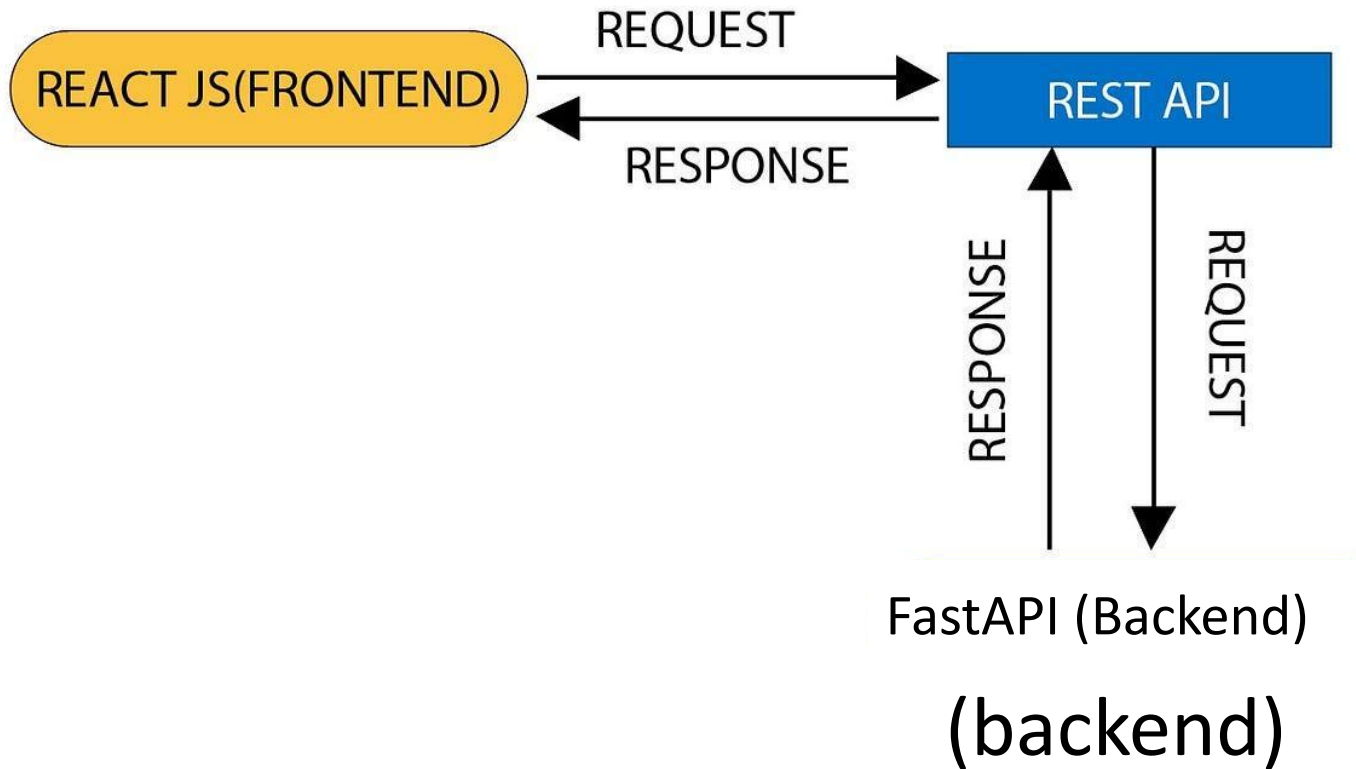
Cloud Computing/AWS



WEB Application Architecture



WEB Application Architecture

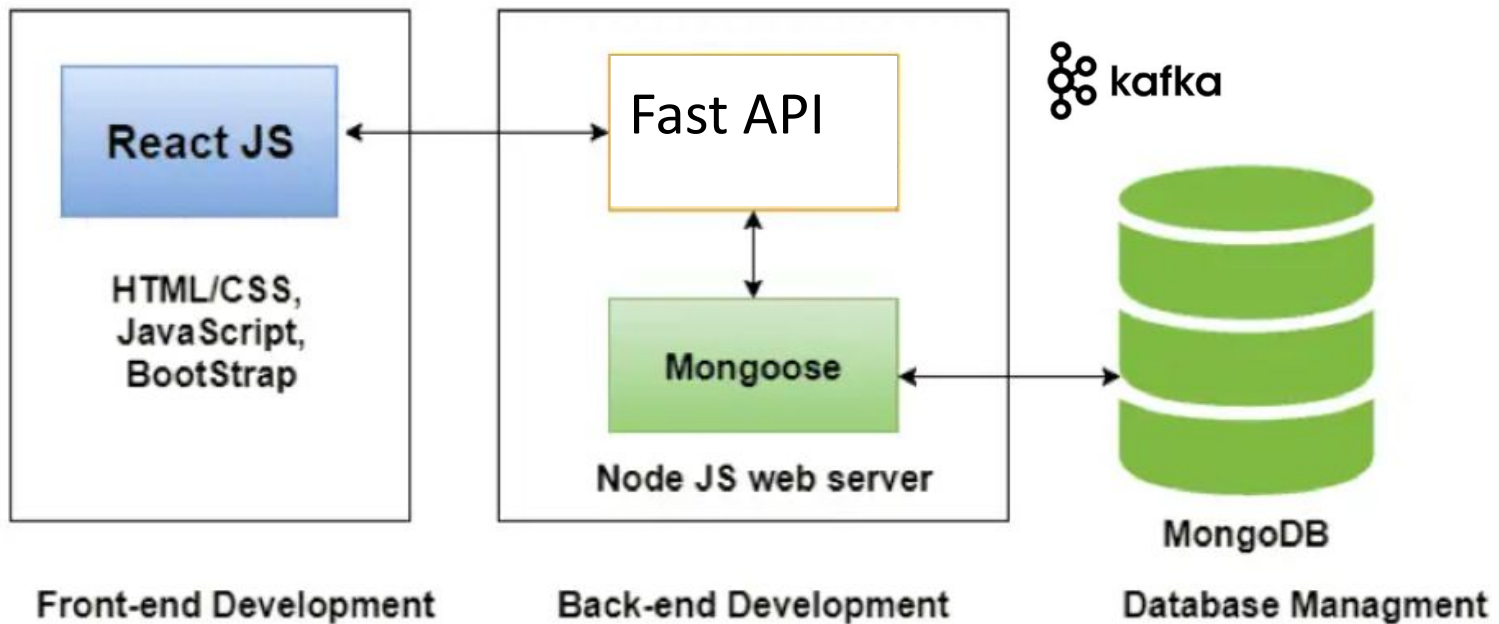


Stack

- MongoDB(M)
 - React(R)
 - FastAPI
-
- Distributed using Kafka message

Stack + Kafka

- MongoDB, FastAPI are dedicated to developing the back-end of web applications. This corresponds to database management, scripts, html documents, HTTP requests, etc.
- React's role is to execute HTTP requests.



Why Kafka? Synchronous vs Event-Driven

- Synchronous (REST): the caller waits for the reply
 - Simple to reason about, easy to debug
 - But the caller is blocked, and if the callee is down the request fails
- Asynchronous (Kafka): the producer writes an event and moves on
 - Consumers read the event when they are ready
 - Producer and consumer need not be running at the same time
- Use it when work is slow, retryable, or must fan out to several services. Do not use it when the user is waiting for the answer.

Frontend Technology

Introduction to web technologies:

- HTML to create the document structure and content
- CSS to control is visual aspect
- Javascript for interactivity



FastAPI agent framework

- We will implement our **Agent AI** capabilities using the [FastAPI Agents framework](#), enabling the seamless creation and management of intelligent agents via REST. This design allows any Node.js components to easily call Python REST services, ensuring tight interoperability between the two backends

Building Agent Endpoints

- Expose Agent Functionality via RESTful APIs: Define API endpoints using FastAPI's decorators (@app.post, @app.get, etc.) that correspond to specific agent actions
- Handle Agent Responses: Define the structure of the API responses (outputs from your agent) using Pydantic models, enabling consistency and predictable data exchange.
- Asynchronous Operations: Leverage FastAPI's async/await features to handle potentially I/O-bound tasks that agents often perform allowing for non-blocking execution and high throughput.

Tools

What do we need to start:

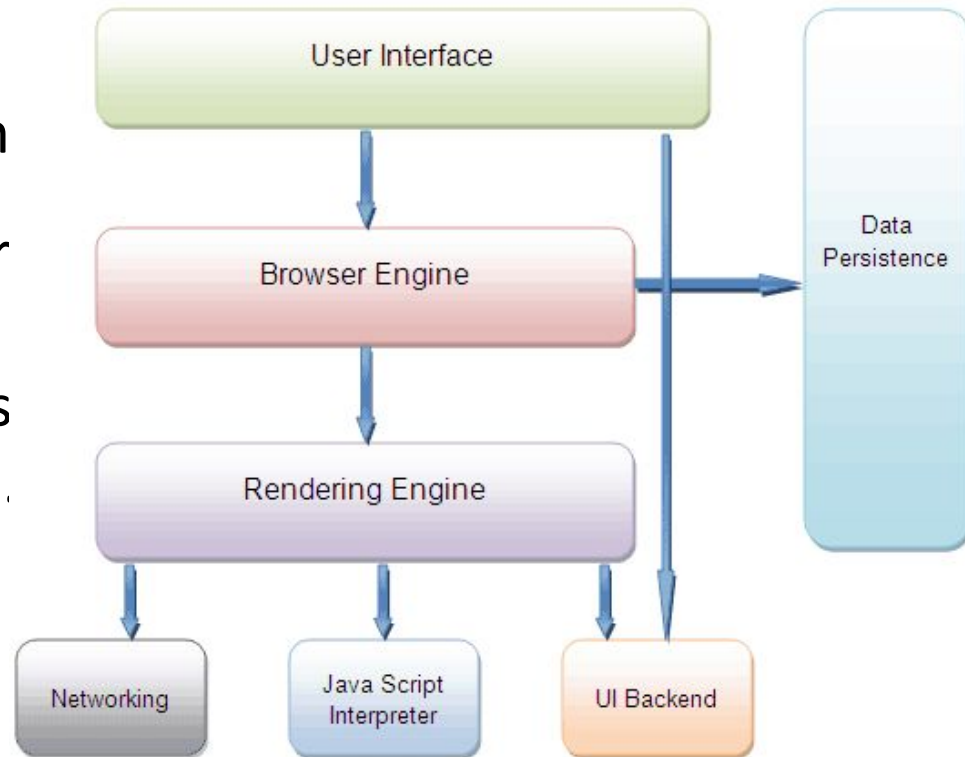
- a good web-browser (Chrome or Firefox)
- a good text editor like:
 - [Notepad++](#) (win)
 - [VSCode](#) (cross platform)
 - textWrangler (osx)
 - [sublime text](#) (cross platform)

Inside a browser

Browsers have parts.

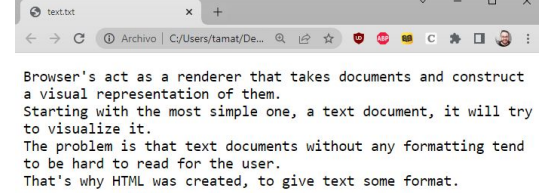
We are interested in two of them

- the Rendering Engine (in charge of converting HTML and CSS into a visual image).
- The Javascript Interpreter (also executing the Javascript code).



Frontend Technologies

- HTML
- CSS
- Javascript



Browsers as a renderer

- Browser's act as a renderer that takes documents and construct a visual representation of them.
- Starting with the most simple one, a text document, it will try to visualize it.
- You can try, drop any .txt file into your browser to visualize it.
- The problem is that text documents without any formatting tend to be hard to read for the user (and quite boring).
- That's why HTML was created, to give text some format.

Markup language

There are many markup languages that add special tags into the text that the renderer won't show but use to know how to display the text.

In HTML these tags use the next notation:

My name is `<b>Javi</b>`



HTML

HTML means Hyper Text Markup Language.

```
<html>
  <head>
  </head>
  <body>
    <div>
      <p>Hi</p>
    </div>
  </body>
</html>
```

The HTML allow us to define the structure of a document or a website.

HTML is to give structure to the content of the website, not to define an algorithm.

It uses [XML](#) syntax

HTML: basic rules

Some rules about HTML:

- It uses XML syntax (tags with attributes, can contain other tags).

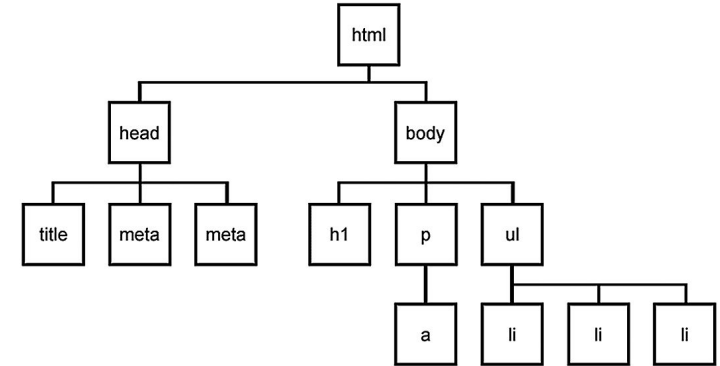
`<tag_name attribute="value"> content`

`</tag_name>`

- There are different HTML elements for different types of information and behaviour.
- The information is stored in a tree-like structure (nodes that contain nodes inside) called DOM (Document Object Model).
- It gives the document some semantic structure (pe. this is a title, this is a section, this is a form) which is helpful for computers to understand websites content.
- It must not contain information related to how it should be displayed (that information belongs to the CSS), so no color information, font size, position, etc.

DOM is a tree

Every node can only have one parent, and every node can have several children, so the structure looks like a tree.





HTML: main tags

Although there are lots of tags in the HTML specification, 99% of the webs use a subset of HTML tags with less that 10 tags, the most important are:

- `<div>`: a container, usually represents a rectangular area with information inside.
- `<img/>`: an image
- `<a>`: a clickable link to go to another URL
- `<p>`: a text paragraph
- `<h1>`: a title (h2,h3,h4 are titles of less importance)
- `<input>`: a widget to let the user introduce information
- `<style>` and `<link>`: to insert CSS rules
- `<script>`: to execute Javascript
- `<span>`: a null tag (doesn't do anything), good for tagging info

HTML: other interesting tags

There are some tags that could be useful sometimes:

- `<button>`: to create a button
- `<audio>`: for playing audio
- `<video>`: to play video
- `<canvas>`: to draw graphics from javascript
- `<iframe>`: to put another website inside ours

How to learn HTML/CSS

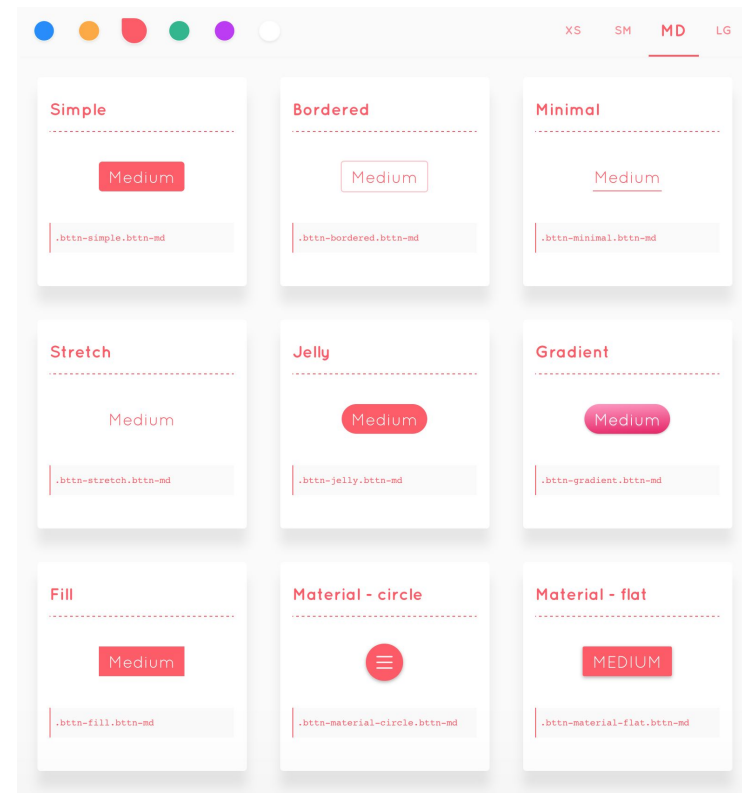
- <https://www.w3schools.com/>
- [HTML Reference](#): a description of all HTML tags.
- [The 25 Most used tags](#): a list with information of the more common tags.
- [HTML5 Good practices](#): some tips for starters

CSS

CSS allows us to specify how to present (render) the document info stored in the HTML.

Thanks to CSS we can control all the aspects of the visualization and some other features:

- Colors: content, background, borders
- Margins: interior margin, exterior margin
- Position: where to put it
- Sizes: width, height
- Behaviour: changes on mouse over



CSS example

```
* {  
    color: blue; /*a comment */  
    margin: 10px;  
    font: 14px Tahoma;  
}
```

This will change all the tags in my web (‘*’ means all) to look blue with font Tahoma with 14px, and leaving a margin of 10px around.

CSS fields

Here is a list of the most common CSS fields and an example:

- `color: #FF0000; red; rgba(255,00,100,1.0);` //different ways to specify colors
- `background-color: red;`
- `background-image: url('file.png');`
- `font: 18px 'Tahoma';`
- `border: 2px solid black;`
- `border-top: 2px solid red;`
- `border-radius: 2px;` //to remove corners and make them more round
- `margin: 10px;` //distance from the border to the outer elements
- `padding: 2px;` //distance from the border to the inner elements
- `width: 100%; 300px; 1.3em;` //many different ways to specify distances
- `height: 200px;`
- `text-align: center;`
- `box-shadow: 3px 3px 5px black;`
- `cursor: pointer;`
- `display: inline-block;`
- `overflow: hidden;`

CSS how to add it

There are four ways to add CSS rules to your website:

- Inserting the code inside a style tag

```
<style>
  p { color: blue }
</style>
```

- Referencing an external CSS file

```
<link href="style.css" rel="stylesheet" />
```

- Using the attribute style on a tag

```
<p style="color: blue; margin: 10px">
```

- Using Javascript (we will see this one later).

CSS selectors

Let's start by changing the background color of one tag of our website:

```
div {  
    background-color: red;  
}
```

This CSS rule means that every tag DIV found in our website should have a red background color. Remember that DIVs are used mostly to represent areas of our website.

We could also change the whole website background by affecting the tag body:

```
body {  
    background-color: red;  
}
```


CSS further reading

- <https://www.w3schools.com/>
- [One line layouts tutorials](#)
- [Understanding the Box Model](#): a good explanation of how to position the information on your document.
- [All CSS Selectors](#): the CSS selectors specification page.
- [CSS Transition](#): how to make animations just using CSS
- [TailwindCSS](#): a CSS Framework

What is JavaScript?

JavaScript is a programming language used to make websites interactive and dynamic. It was originally created by Netscape for use in web browsers and is now standardized as ECMAScript.

Key ideas:

- Runs mainly in the browser to control page behavior and respond to user actions
- Can update HTML/CSS, validate forms, create animations, and handle events
- Standardized as ECMAScript
- Also runs on servers through platforms like Node.js

Executed by a JavaScript engine

- Browsers such as Chrome, Firefox, and Safari contain JavaScript engines that execute JavaScript code.
- Traditionally JavaScript was described as an **interpreted language**, because source code could be executed directly without the programmer compiling it first.
- Modern JavaScript engines actually use techniques such as **just-in-time (JIT) compilation** to improve performance.

JavaScript vs. Java

- JavaScript is usually interpreted or just-in-time compiled by the browser
- Java programs are typically compiled and can run as standalone applications

How does it work?

- Embedded within HTML page
- Simple programming statements combined with HTML tags
- Interpreted

Learning JavaScript

- Special syntax to learn
- Learn the basics and other samples
- Write it in IDE, view results in browser

Including JavaScript in HTML

- Two ways to add JavaScript to Web pages
 - Use the `<script>...</script>` tag
 - Include the script in an external file -- more about this later in the semester
- Initially, we will only use the `<script>...</script>` tag

JavaScript keywords

break case catch continue debugger
default delete do else finally
for function if in instanceof
new return switch this throw
try typeof var void while
with

•Reserved words

class const enum export extends
import implements interface let package
private protected public static super yield

JavaScript Tutorial

W3school: <https://www.w3schools.com/js/>

The Modern JavaScript Tutorial: <https://javascript.info/>

JavaScript tutorial: <https://www.javascripttutorial.net/>

Containers and Deployment:

What is Docker?

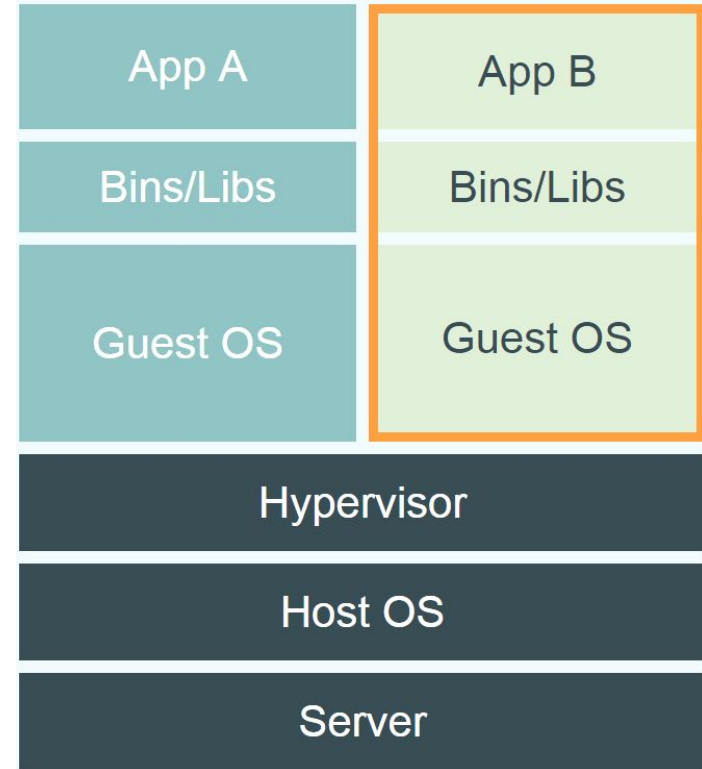
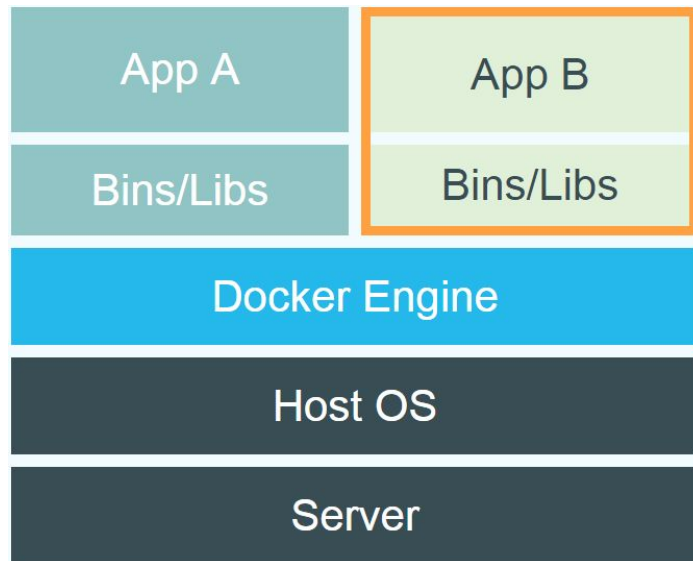
- **Docker** is an open platform for **developing, shipping,** and **running** applications.
- **Docker** is designed to deliver your applications faster.
- Docker you can separate your applications from your **infrastructure** and treat your infrastructure like a managed application

Docker



- Provide a uniformed wrapper around a software package: «*Build, Ship and Run Any App, Anywhere*» [www.docker.com]
 - Similar to shipping containers: The container is always the same, regardless of the contents and thus fits on all trucks, cranes, ships, ...

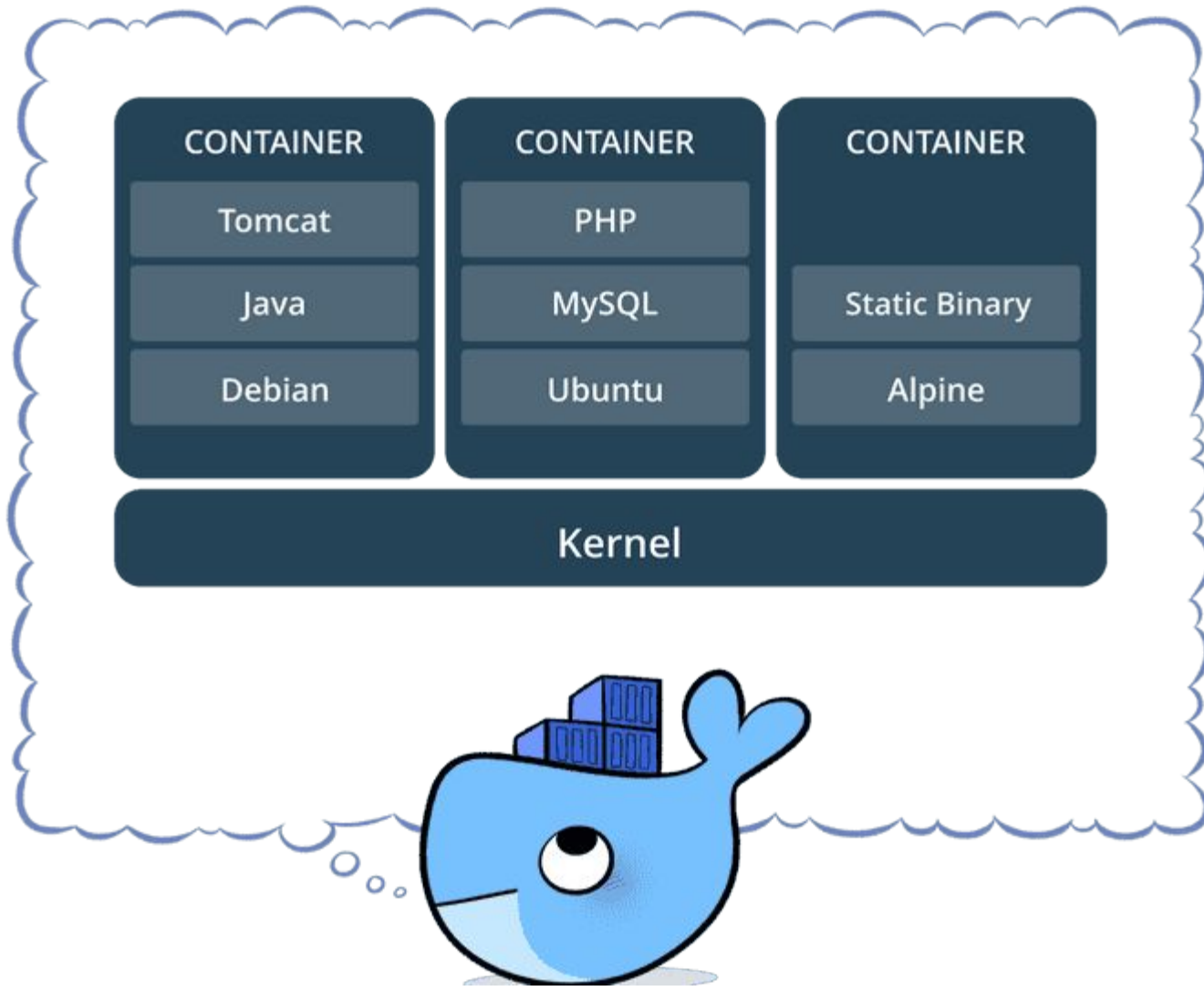
Docker vs. Virtual Machine



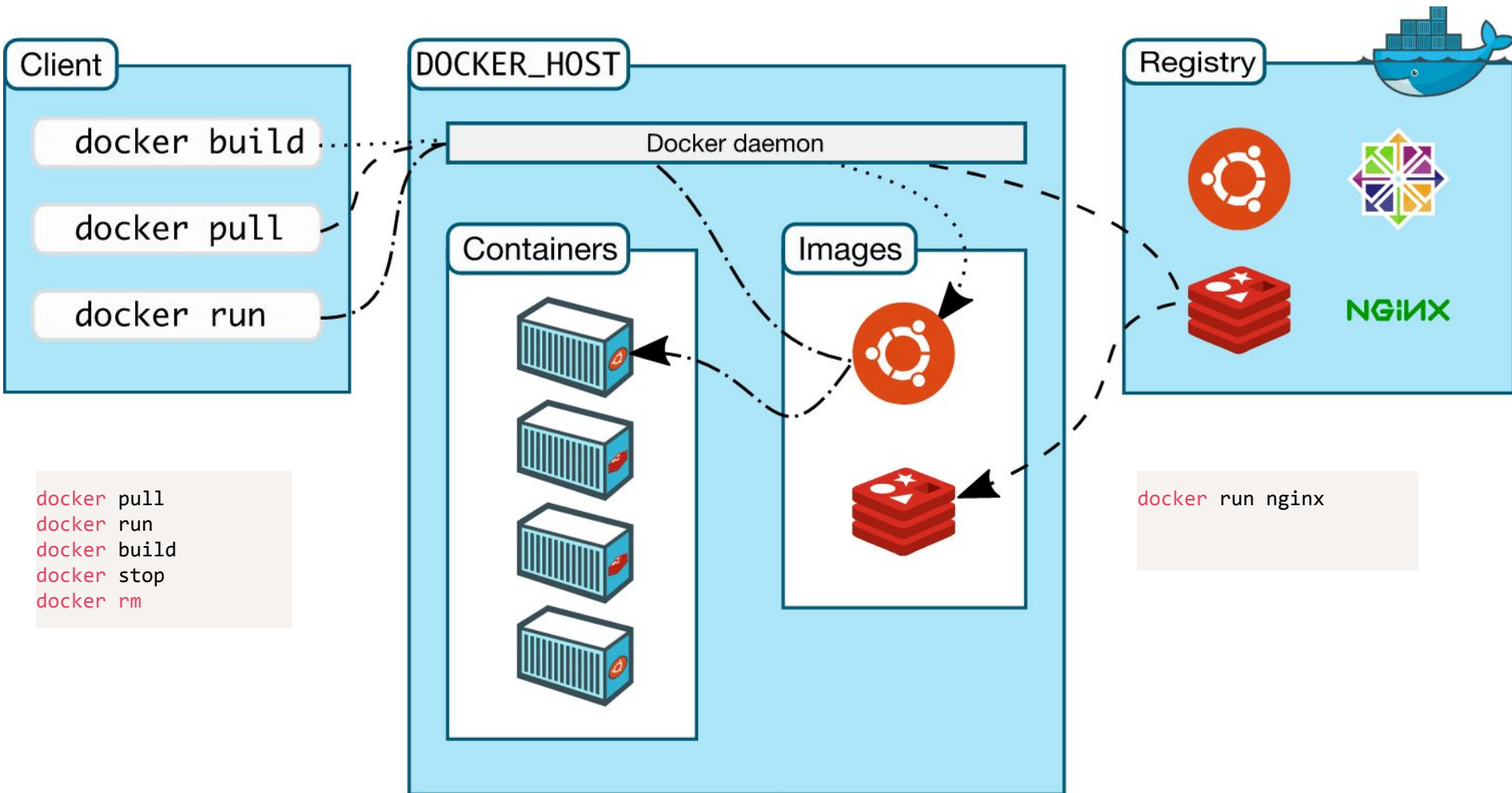
Docker Features

- fast and elegant isolation framework
- Inexpensive
- Low CPU/memory overhead
- Fast boot/shutdown
- Cross cloud infrastructure

Docker container

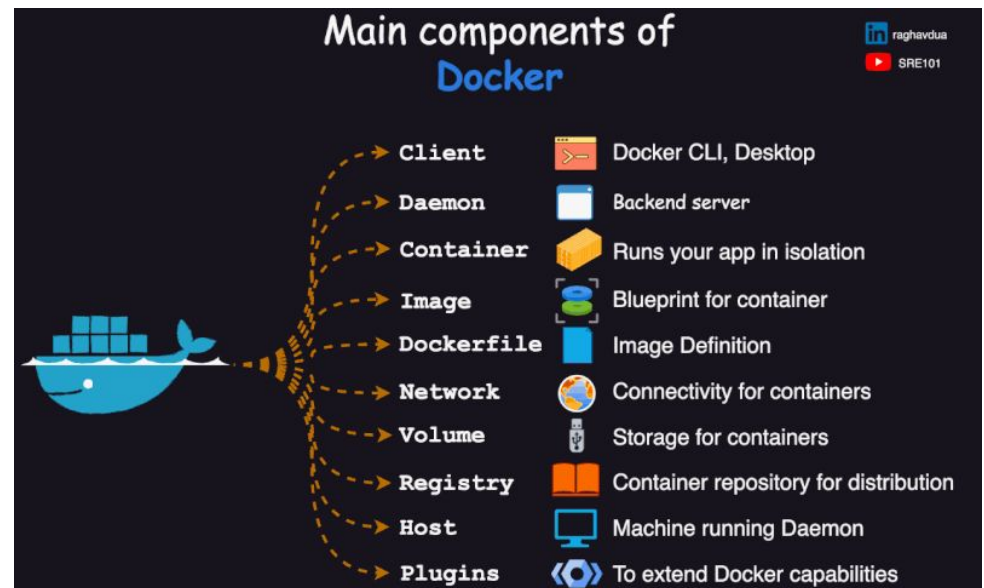


Docker Architecture



Docker Components

- Docker Client
- Docker Daemon
- Docker Registry
- Docker Containers
- Docker Images
- Dockerfile



Run Applications

- Step 1:** Build an image.
- Step 2:** Run the container.
- Docker Image is a read-only template to build containers. An image holds all the information needed to bootstrap a container, including what processes to run and the configuration data. Every image starts from a base image, and a template is created by using the instructions that are stored in the DockerFile. For each instruction, a new layer is created on the image.
- Running the container originates from the image we created in the previous step. When a container is launched, a read-write layer is added to the top of the image. After appropriate network and IP address allocation, the desired application can now be run inside the container.

Docker Use Cases

- Development Environment
- Environments for Integration Tests
- Quick evaluation of software
- Microservices
- Multi-Tenancy
- Unified execution environment (dev ☐ test ☐ prod (local, VM, cloud, ...))

AI Agents: What is an Agent?

An **Agent** is a system designed to achieve goals through an iterative cycle of reasoning and interaction with its environment.

Core Process

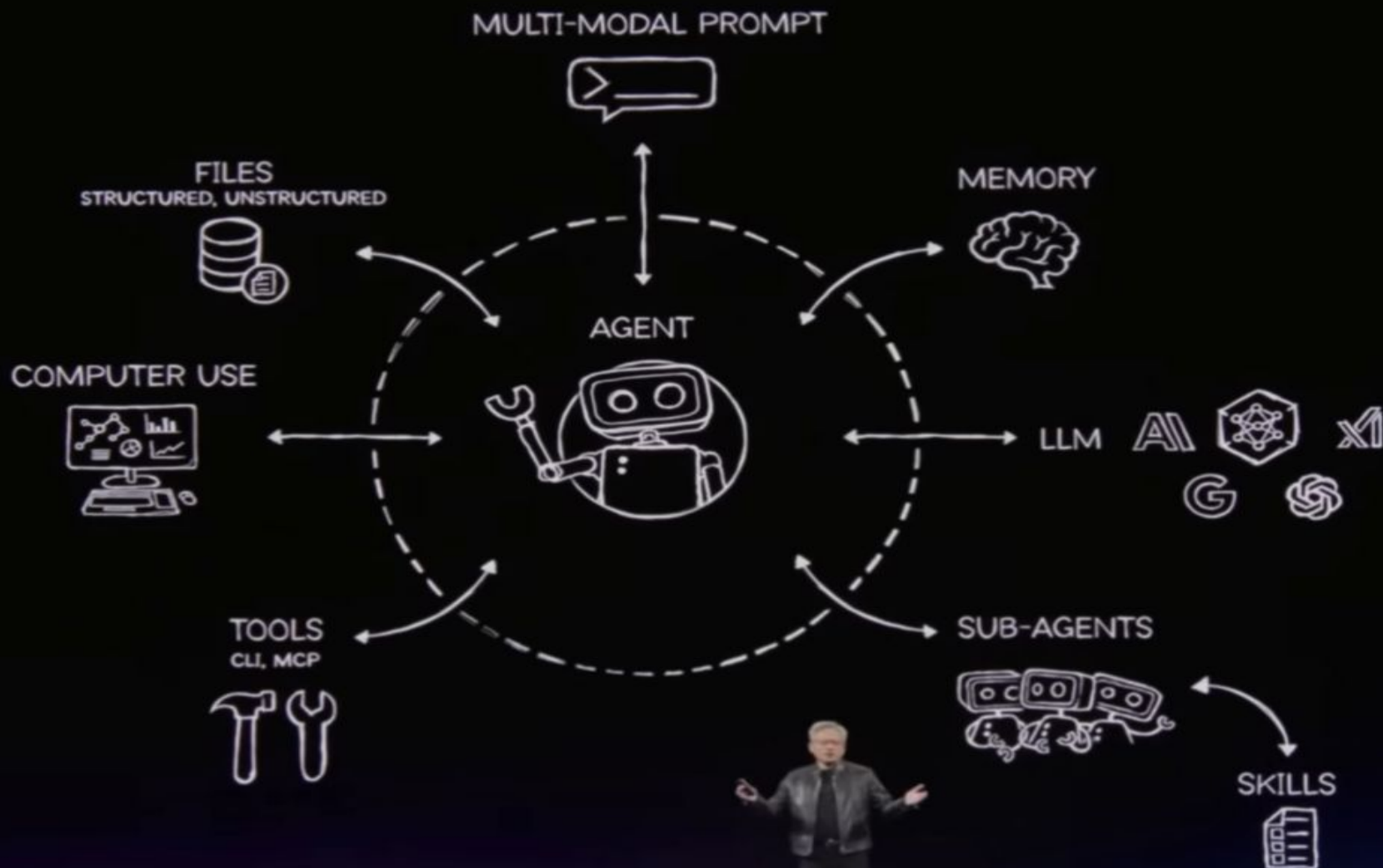
1. **Plan** → Formulate actions toward a goal.
2. **Act** → Execute actions, often by using external tools or functions.
3. **Observe** → Perceive feedback or results from the environment.
4. **Reflect** → Update internal state and strategy based on outcomes.

This loop repeats until the goal is reached (or the task terminates).

Key Components

- **LLM “Brain”** – Provides reasoning, planning, and language understanding.
- **Tools / Function Calls** – Extend capabilities (e.g., search, API calls, computation).
- **Memory / State** – Stores context, history, and learned knowledge for continuity.
- **Control Loop** – Governs the cycle of planning, acting, observing, and reflecting.

Agents – A New Computing Platform



What are AI Agents?

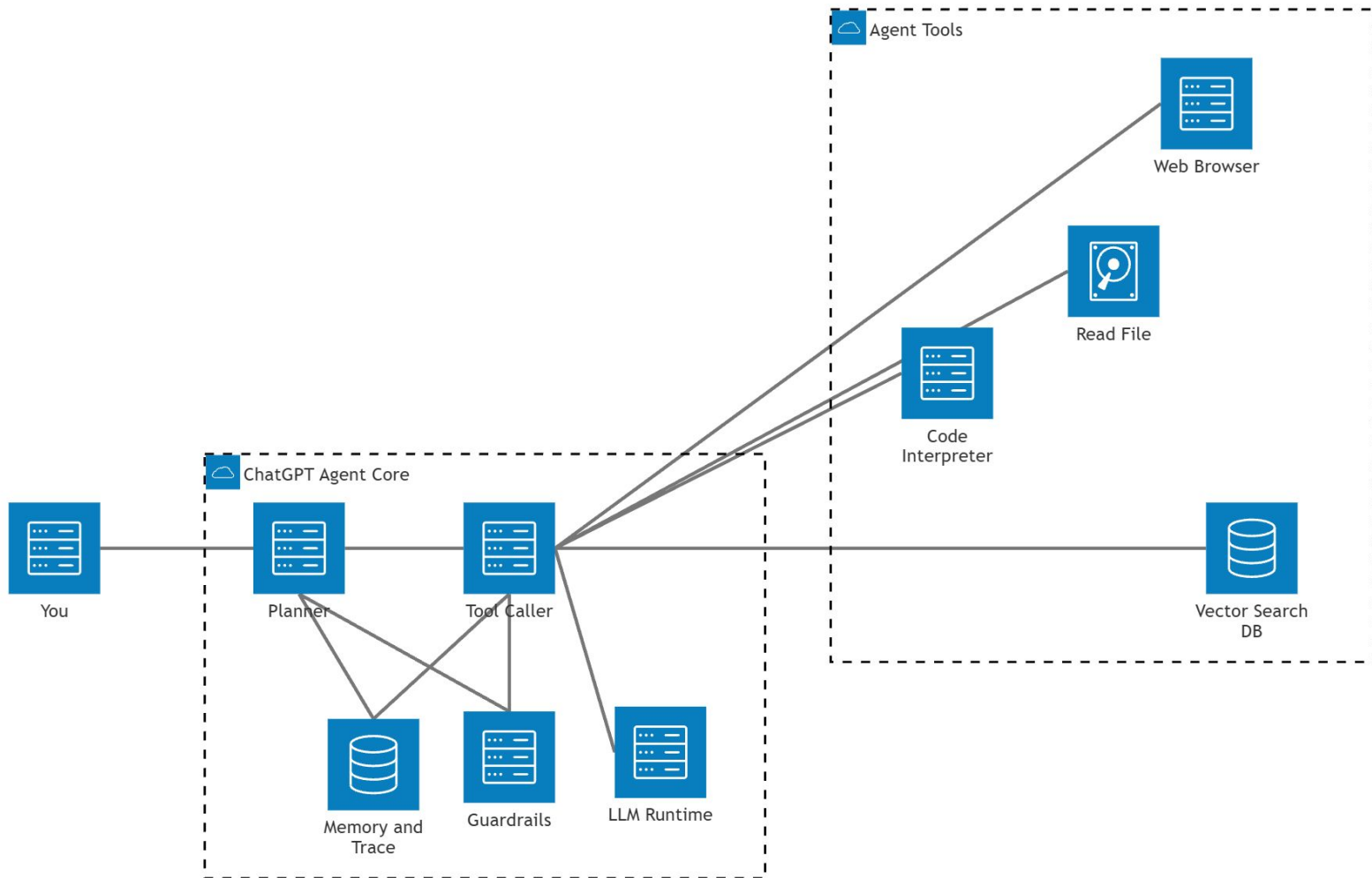
Architecture



How agents work

- **Loop:** Plan → Select tool → Call tool → Observe → Reflect / Update plan → Stop
- **Brain:** LLM/model drives reasoning and selects next steps, Generates tool calls with arguments
- **Tools:** Defined with strict I/O schemas, Reduce errors and hallucinations
- **Memory:** short-term trace + optional long-term vectors
- **Stop conditions:** success/timeout/iteration limits
- *Course reading:* **Introduction to smolagents**

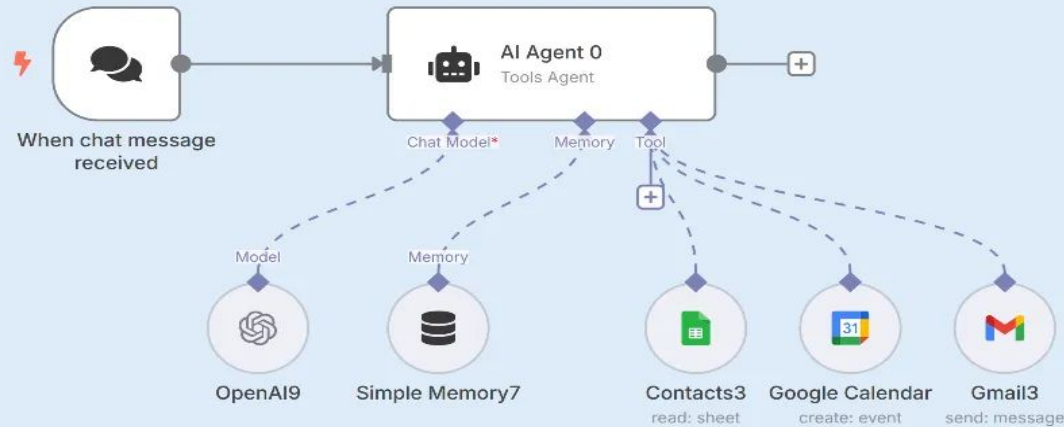
<https://huggingface.co/learn/agents-course/en/unit2/smolagents/introduction>



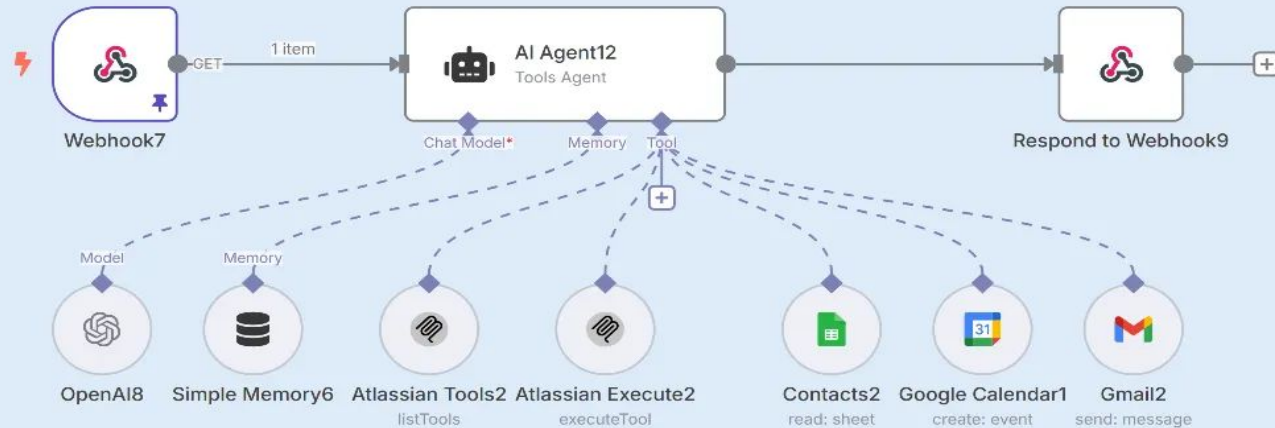
AI Agent Architectures:

<https://www.productcompass.pm/p/ai-agent-architectures>

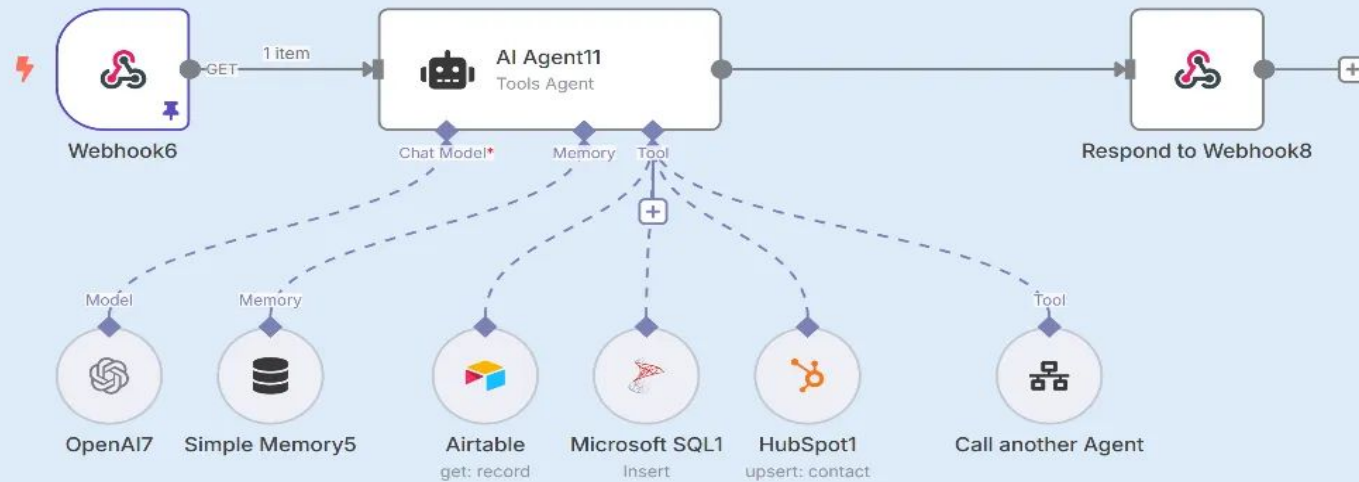
Single Agent + Tools



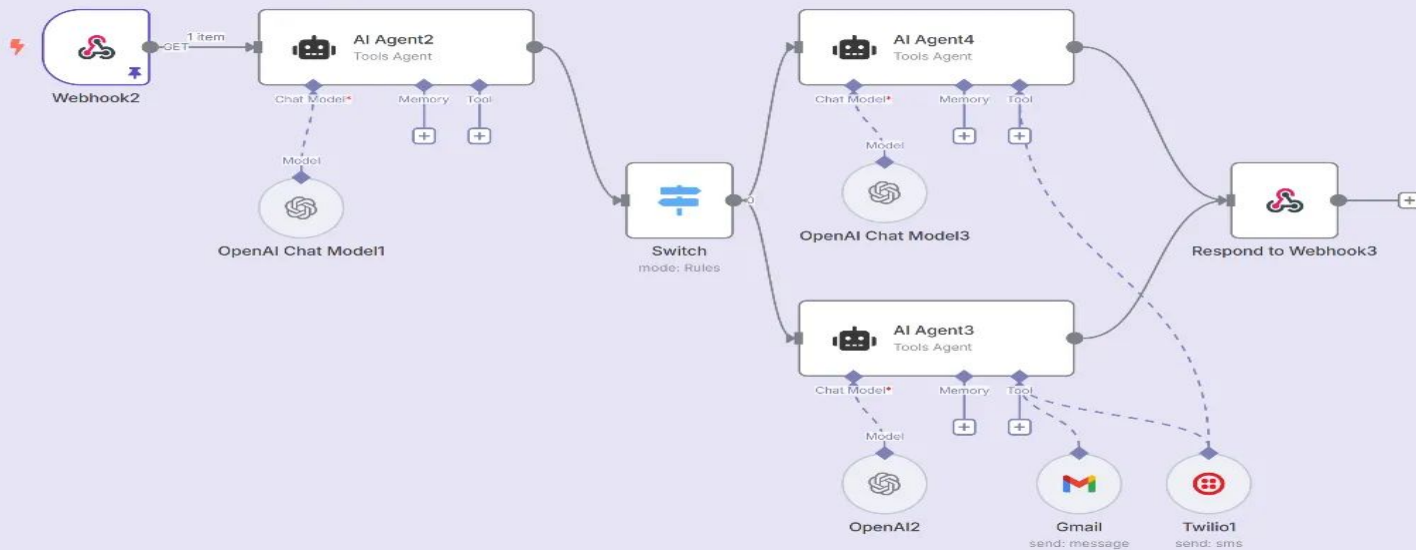
Single Agent + MCP Servers + Tools



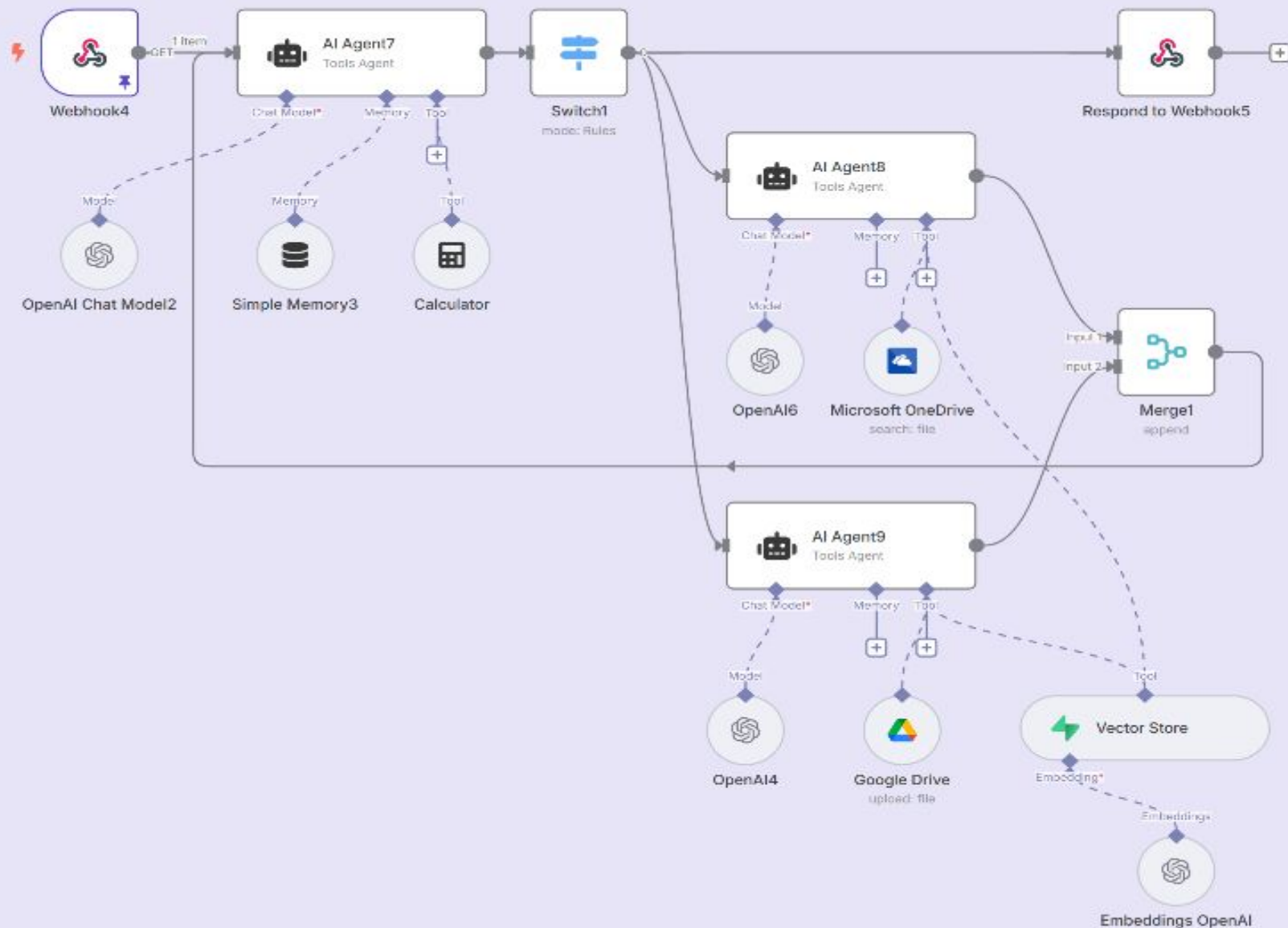
Single Agent + Dynamically Call Other Agents



Agents Hierarchy + Shared Tools + Parallel



Agents Hierarchy + Loop + Parallel + Shared RAG



Tools and functions (schemas matter)

- Tools are **typed functions** (name, inputs, outputs) exposed to the model
Name – identifier for the model (e.g., search_web)
Inputs – required arguments (e.g., {query: string})
Outputs – expected return values (e.g., {results: list of strings})
- Clear contracts $\Rightarrow$ higher reliability, safer actions, easier
Example: A calculator tool guarantees numeric inputs/outputs only.
- Start with a **small allowlist** and expand deliberately
Example: Begin with “search” + “summarize,” later add “email” or “database query.”
- **Schemas:** JSON or YAML definitions that describe tool capabilities

```
{  
  "name": "calculator",  
  "description": "Perform basic arithmetic operations",  
  "inputs": { "expression": "string" },  
  "outputs": { "result": "number" }  
}
```

Core patterns you'll use

Tool use — Agents invoke external functions with strict schemas.

Example:

- **Tool:** `weather_lookup({location: "San Jose"})`
- **Output:** `{temperature: 72, condition: "sunny"}`

Planning — Breaking down a complex goal into **steps** or a **graph of subtasks**.

Example: Goal: “*Book a trip to NYC.*”

Plan: (1) Search flights → (2) Compare prices → (3) Book hotel → (4) Build itinerary.

Reflection — Model **self-critiques** outputs and retries when errors occur.

Example: Agent generates Python code → fails on runtime error.

Reflection: “Error: division by zero. Retry with corrected formula.”

Handoffs — Decide when to **delegate tasks to specialists** vs keeping one generalist agent.

Single-agent: A general assistant answering FAQs + doing simple math.

Multi-agent: *Research Agent* → gathers sources, *Summarizer Agent* → condenses text, *Writer Agent* → drafts final report

Memory that matters

Short-term: Keeps track of the **last few turns** of context, Example:

`ConversationBufferWindowMemory(k=6)` → model “remembers” the last 6 exchanges. Useful for: chit-chat, short tasks, avoiding overload.

Episodic: Stores **summarized chunks** of past interactions. Example:

`ConversationSummaryMemory` → auto-generates summaries of earlier sessions. Useful for: keeping track of multi-session progress without recalling every detail.

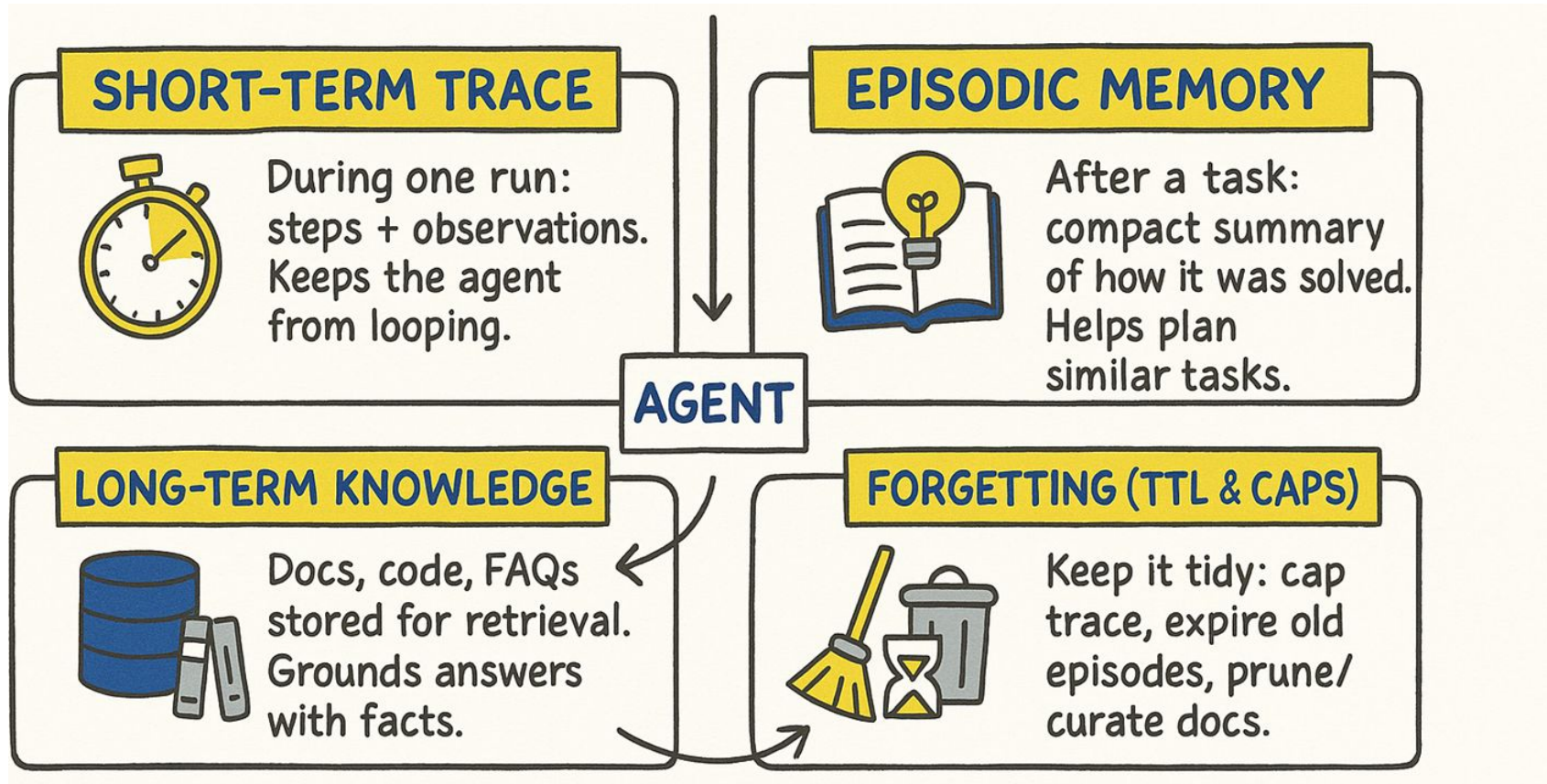
Long-term: Uses **vector databases** to store and retrieve knowledge over time.

Example: `FAISS + HuggingFaceEmbeddings + retriever/tool` → semantic search across prior conversations, documents, or knowledge base. Useful for: recalling facts, prior work, personal history.

Forgetting: **Windowed memory** (`k`) → discards oldest turns. **Episodic TTL (time-to-live)** → optional expiration of memories after set time. Useful for: controlling memory growth, ensuring relevance.

Example in action:

- User: *“Remind me what we discussed about robotics last week.”*
- Short-term memory → no record.
- Episodic memory → summary: *“User explored simulation platforms.”*
- Long-term memory → retrieves FAISS entry with exact notes: *“We used Habitat and iGibson for navigation tasks.”*



Evaluating an Agent

- Non-deterministic systems cannot be tested with `assertEqual` alone
- What to measure
 - Task success rate - did it finish the job, judged against a fixed set of cases
 - Steps per task - a rising step count usually means confusion
 - Tool-call accuracy - right tool, right arguments
 - Cost per task - tokens x price, summed over every loop iteration
 - Latency, p50 and p95 - agents are slow, and users notice the tail

Evaluating an Agent

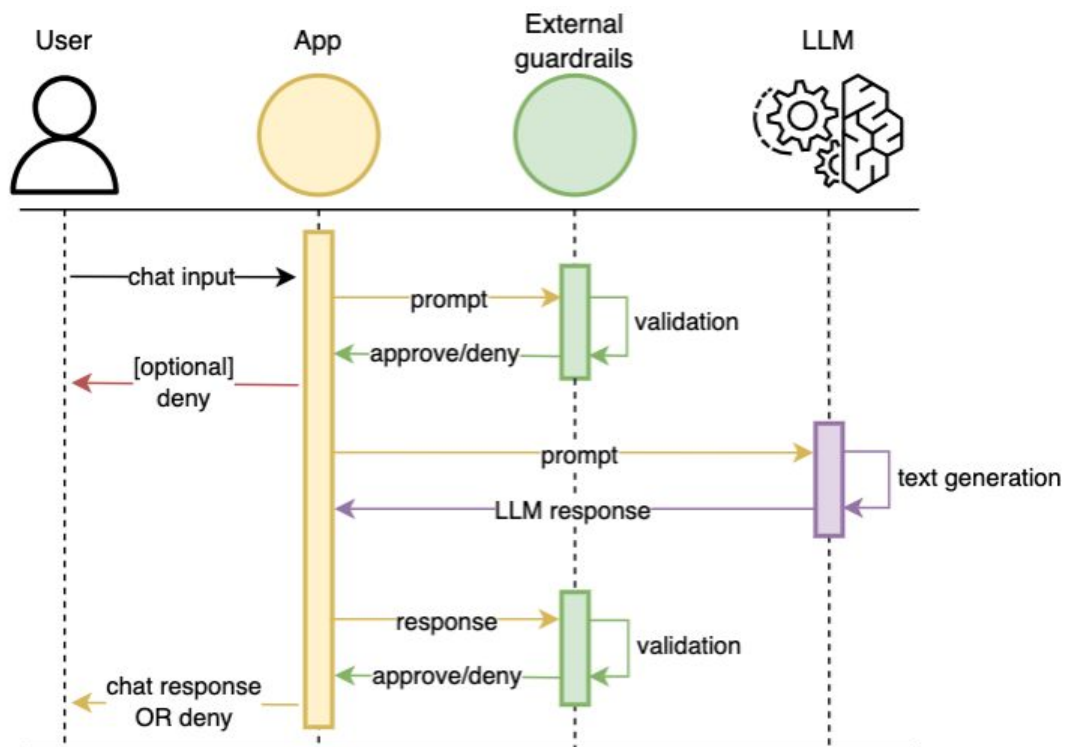
- Practical habits
 - Build a small evaluation set early: 20 to 50 realistic tasks
 - Log every step - prompt, tool call, observation, decision
 - Always compare against a simpler baseline (single prompt, or RAG)
- If the agent does not beat the baseline, ship the baseline.

Guardrails & safety (day-one essentials)

Before action: tool allowlist, argument validation, schema checks

During: sandboxed code exec, timeouts, rate limits, secrets hygiene

After: audits/logs, cost & latency budgets, optional human review



When not to use an agent

Use a plain workflow / function call if...

The task is **deterministic & verifiable** (fixed steps, exact output).

Agents add

cost/latency for flexibility you don't need.

There's a **pre-defined flow** that already handles requests reliably (don't hand

You need **repeatable results** (compliance, audits). LLMs are **non-deterministic**

You're under strict **SLA/latency or cost** constraints where extra LLM steps aren't

Rule of thumb: **start simple** (single LLM + retrieval). Only add agent loops when simpler options fail.

MS AI Doctor

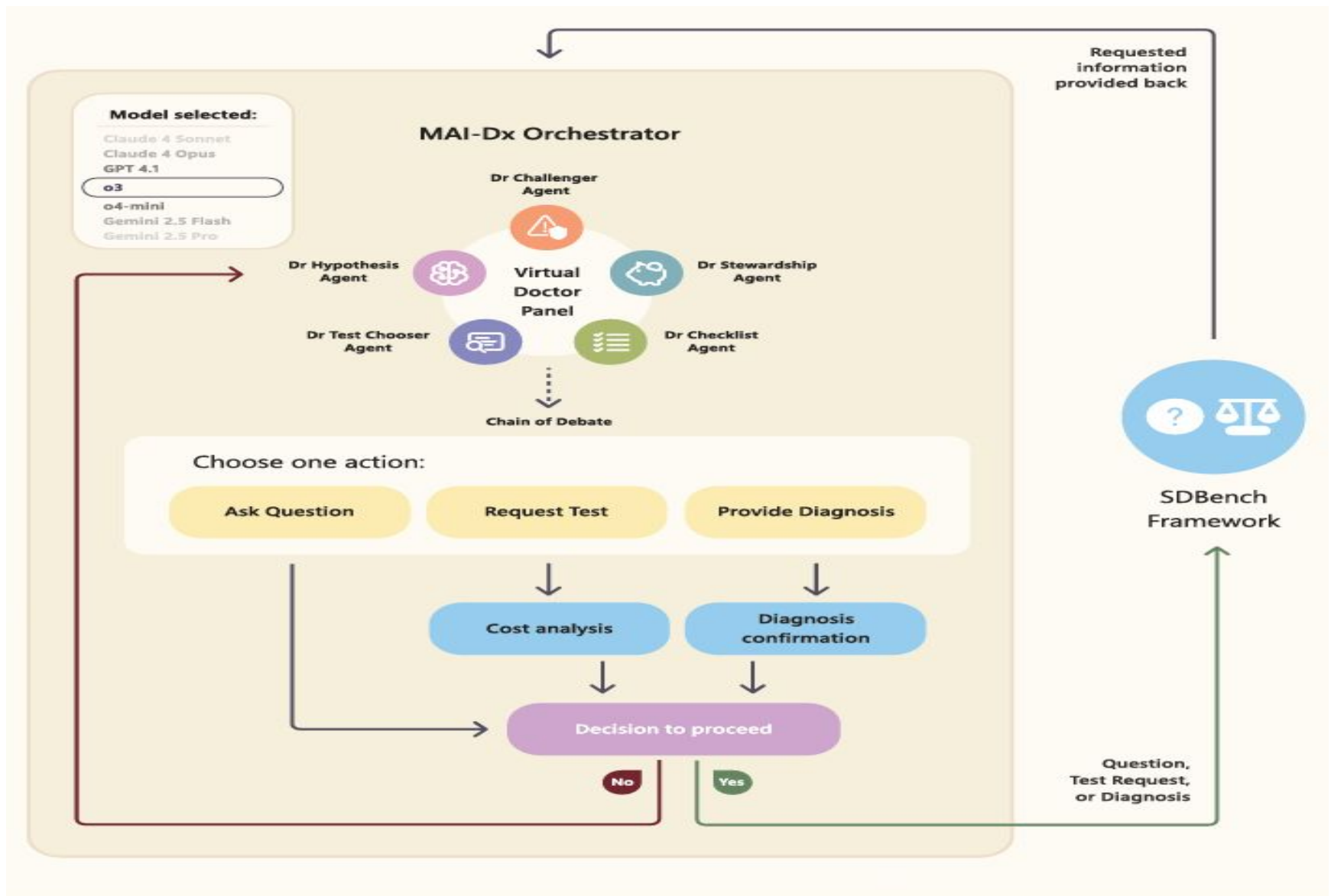


Figure 5: Overview of the MAI-Dx Orchestrator

AI Engineer?

AI Engineers build, test, and deploy AI models, as well as maintain the underlying AI infrastructure. They are problem-solvers who can navigate between traditional software development and machine learning implementations

AI Engineer



The role

- Machine learning, data science
- Software design
- Create and deploy machine learning algorithms

Background

- Degree in: Computer science, robotics, engineering, physics
- ML Coursera, AI Google Education
- MSc, PhD in related fields

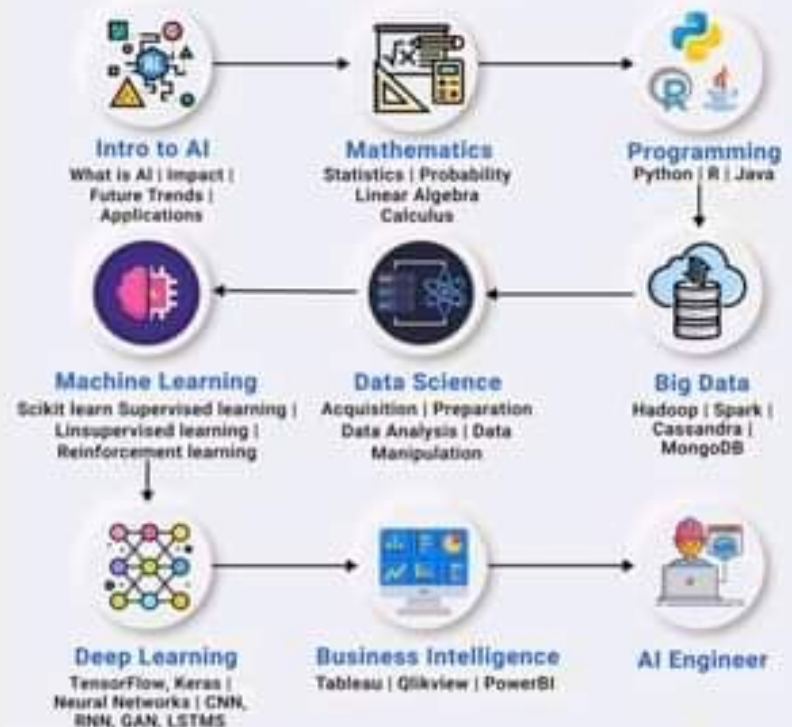
Skills

- Data Science & Statistics
- Mathematics
- CI/CD & SDLC knowledge
- CS & Programming

Salary

Junior: \$ 57,000
Average: \$ 86,000
Top: \$ 114,000

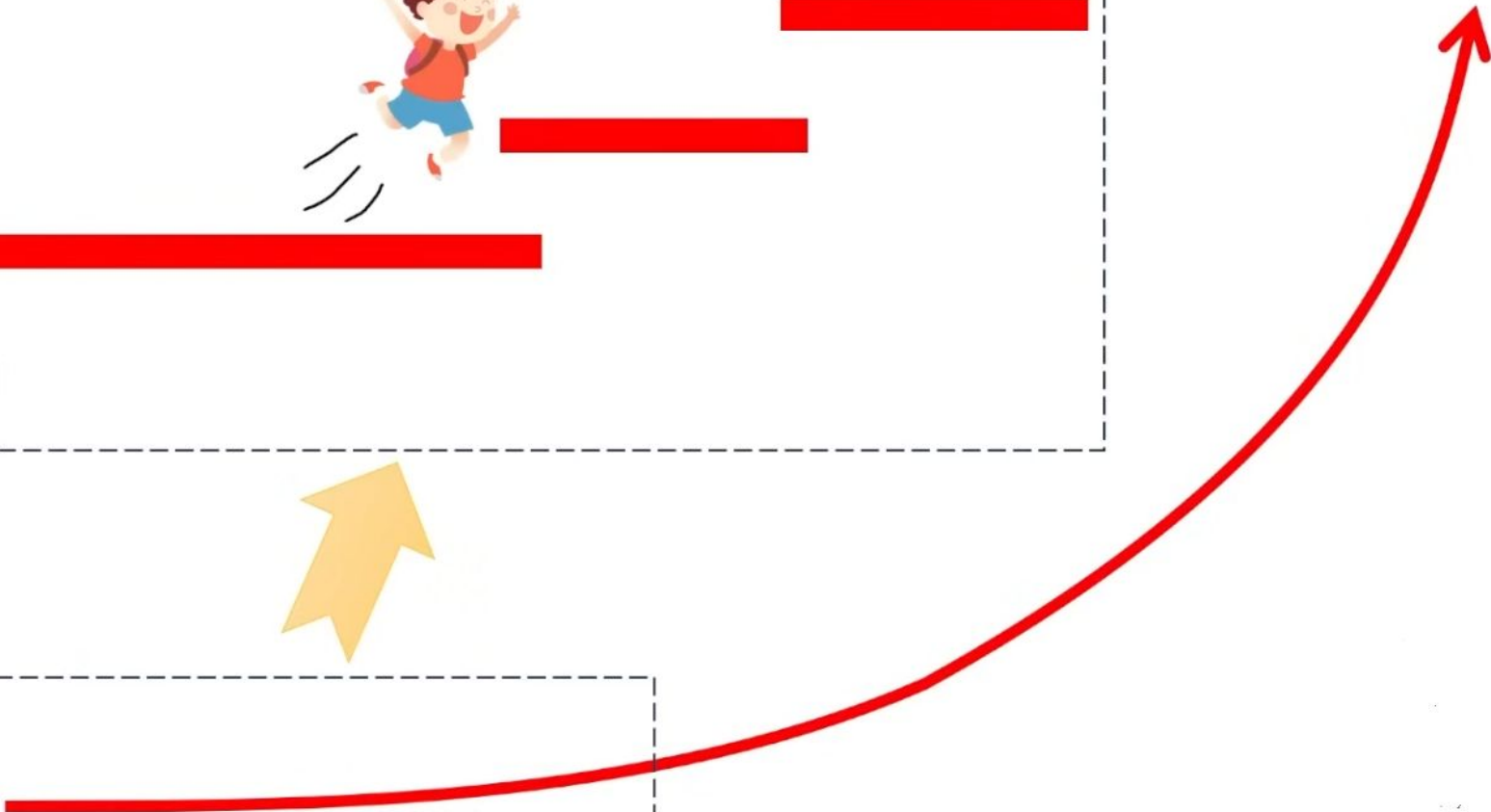
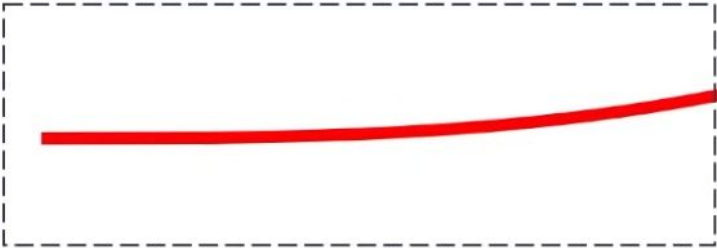
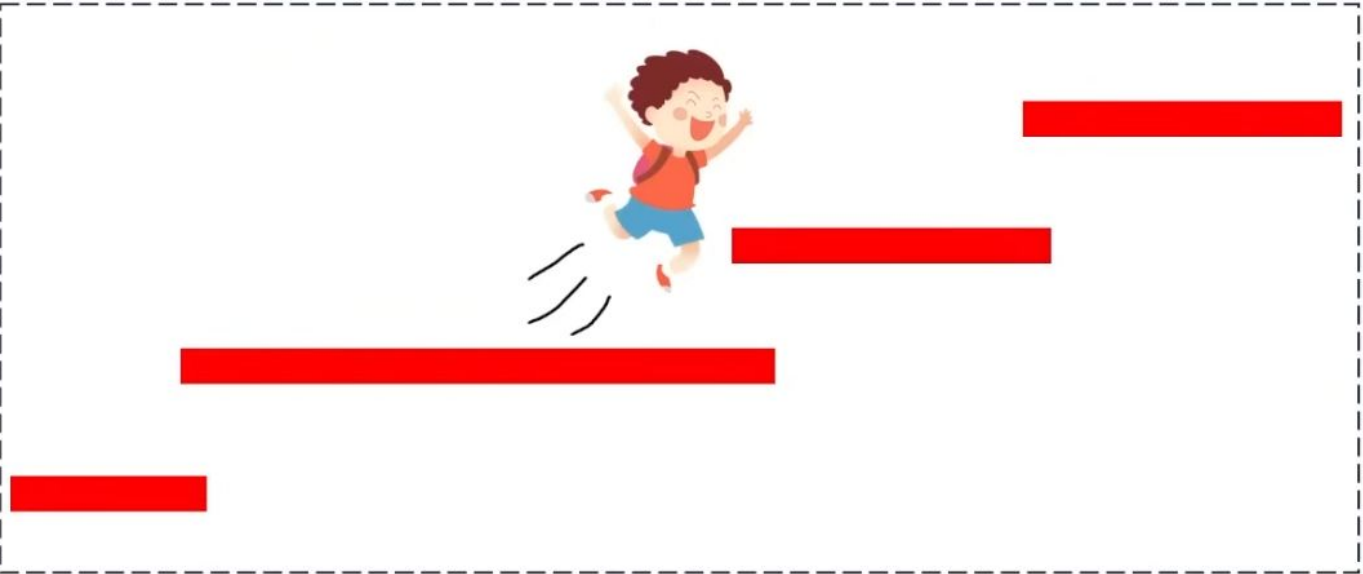
AI ENGINEER ROADMAP



A man with short brown hair and a beard, wearing a black sweater over a red and white checkered shirt, is gesturing with his right hand while speaking. He is positioned on the left side of the frame against a solid green background.

**"There's a new kind
of coding I call "vibe
coding," where you
fully give in to the
vibes."**

-Andrej Karapathy



From Brick Worker to Architect using AI

The “Brick Worker” (Foundational Skills)

- Manual coding, debugging, understanding core concepts.
- Hands-on experience with the “materials” of software.
- Analogy: “Knowing how to lay a brick correctly.”



The “Architect” (AI-Assisted Design)

- Using AI tools for code generation, system design, and automation.
- Focus on high-level structure, integration, and strategy.
- Analogy: “Designing the building.”



The Crucial Insight: Without foundational knowledge (“knowing how to be a brick worker”), you cannot effectively evaluate the quality, security, or efficiency of AI-generated solutions (“tell whether brick work is good or not”)

The most important message is the box at the bottom: **you cannot become a good “architect” without understanding the “brick work.”** AI can generate code, but you still need enough technical knowledge to judge whether that code is correct, secure, efficient, maintainable, and appropriate for the problem.

First: learn how to build and debug things yourself. (Lab 1 and Lab 2)

Then: learn to use AI to build faster. (Group Project)

Finally: learn to evaluate and design systems that may contain a lot of AI-generated work.

Use AI to Learn, Not to Replace Learning

Hands-On Coding Builds Real Understanding

- AI can generate code quickly, but **reading generated code is not the same as knowing how to build it**
- Writing, running, debugging, and modifying code yourself develops the technical foundation needed to understand how a system actually works

Do Not Turn Homework Into Copy-and-Paste

- Do **not** copy the homework prompt into ChatGPT and submit the generated answer as your own work
- If you cannot explain your code or reproduce the results yourself, you have not completed the learning objective

Productive Struggle Is Part of Learning

- Debugging errors, interpreting unexpected results, and figuring out why something failed are essential parts of becoming a strong developer
- Letting AI bypass every difficult step weakens independent problem-solving skills
- Hands-on practice builds the judgment needed to recognize when AI-generated code is wrong, inefficient, or inappropriate

Course Policy using AI-assisted Coding

AI-assisted coding (“vibe coding”) can be a powerful tool, but it should **support—not replace—the development of fundamental programming skills.**

- **Build the foundation first.** Do not rely on AI-assisted coding until you have implemented, run, and debugged similar programs manually **several times.**
- **Homework emphasizes hands-on coding.** Homework assignments and pair-programming activities are designed to strengthen your own coding, debugging, and problem-solving skills (No AI Coding)
- **AI-assisted coding is allowed for the group project.** Once foundational skills have been developed, you may use AI tools more extensively to accelerate development, explore alternatives, and integrate larger systems.
- **You remain responsible for the code.** Whether code is written manually or generated with AI, you must be able to explain, test, modify, and debug it.

Ultimately, the key lies in learning to harness AI

- Boost productivity by automating repetitive tasks.
- Enable innovative solutions previously unattainable.

What language to learn?

Python, data structure/algorithm (takes time)

Javascript (Web)

C (Cuda)

Dear Professor Shim,

My name is Kush Bindal, and I am currently a third-semester student in the Master of Science in Applied Data Intelligence program (previously Data Analytics).

I am writing to express my sincere gratitude for your excellent teaching in the DATA-236 "Distributed Systems" course, which I took last semester (Fall 2023). I thoroughly enjoyed the course; your explanations of complex topics were incredibly clear, and the material itself was fascinating.

I also wanted to share some exciting news. I recently accepted an offer for a Software Development Engineer internship position at Amazon for this upcoming summer!

I genuinely believe the experiences and skills I gained in your class were instrumental in securing this opportunity. In particular, working on the **Uber Eats prototype** and the **Uber Transport simulation** projects was incredibly helpful. These projects not only solidified my understanding of distributed systems concepts but also provided me with concrete examples and scenarios that I was able to draw upon during Amazon's Leadership Principles behavioral round. Discussing the challenges, design choices, and problem-solving involved in those projects allowed me to effectively articulate my experiences and thinking process during my interview.

Thank you again for your dedication to teaching and for designing such practical and impactful projects. Your course has already made a significant difference in my academic and early career journey.

Sincerely,

Kush Bindal

Master of Science in Applied Data Intelligence